

ER Modeling

Chapter # 03

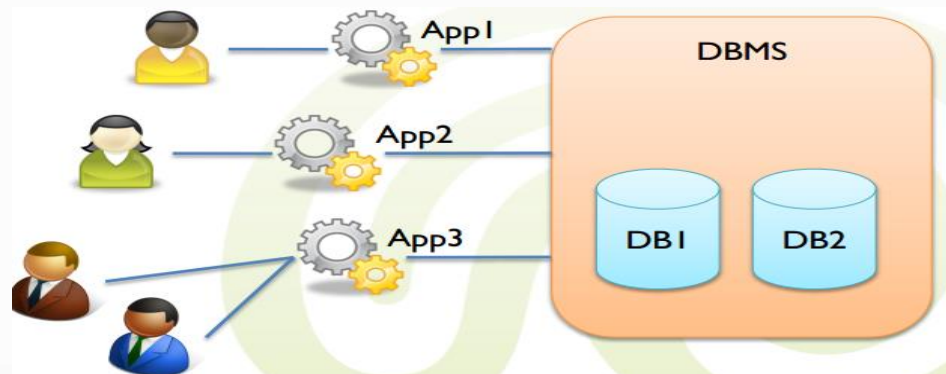
Database Application

Conceptual modeling is a very important phase in designing a successful database application.

Generally, the term database application refers to a particular database and the associated programs that implement the database queries and update

Database applications consist of

- ***database instances with their respective DBMS***
- ***associated application programs interfacing with the users***



Database Application

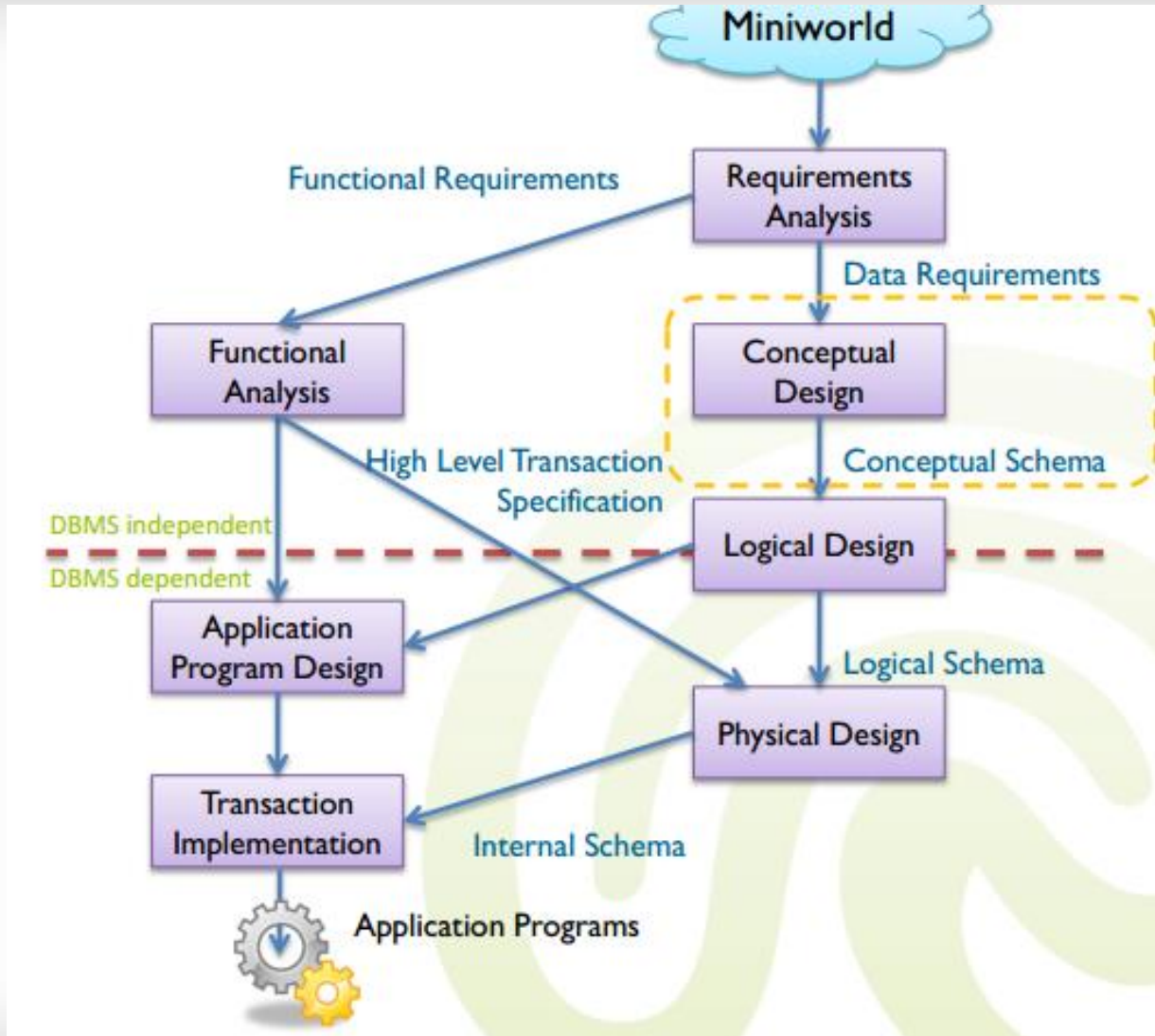
Planning and developing application programs traditionally is a software engineering problem

- ***Requirements Engineering***
- ***Conceptual Design***
- ***Application Design***

Software engineers and data engineers cooperate tightly in planning the need, use and flow of data

- ***Data Modeling***
- ***Database Design***

Phases of DB Design



Phases of DB Design

Requirements Analysis

- database designers interview prospective users and stakeholders
- Data Requirements describe what kind of data is needed
- Functional Requirements describe the operations performed on the data

Functional Analysis

- concentrates on describing high-level user operations and transactions *does not yet* contain implementation details

These consist of the user defined operations(or transactions) that will be applied to the database, including both retrievals and updates

Phases of Database Design

Conceptual Design

- transforms Data Requirements to conceptual model
- describes high-level data entities, relationships, constraints, etc.

The high-level conceptual schema can also be used as a reference to ensure that all users' data requirements are met and that the requirements do not conflict.

This approach enables database designers to concentrate on specifying the properties of the data, without being concerned with storage and implementation details

Phases of Database Design

Logical Design

Logical design is the actual implementation of the database, using a commercial DBMS maps the conceptual data model to the logical data model used by the DBMS

e.g. relational model, hierarchical model

technology independent conceptual model is adapted to the used DBMS software

Physical Design

Creates internal structures needed to efficiently store/manage data

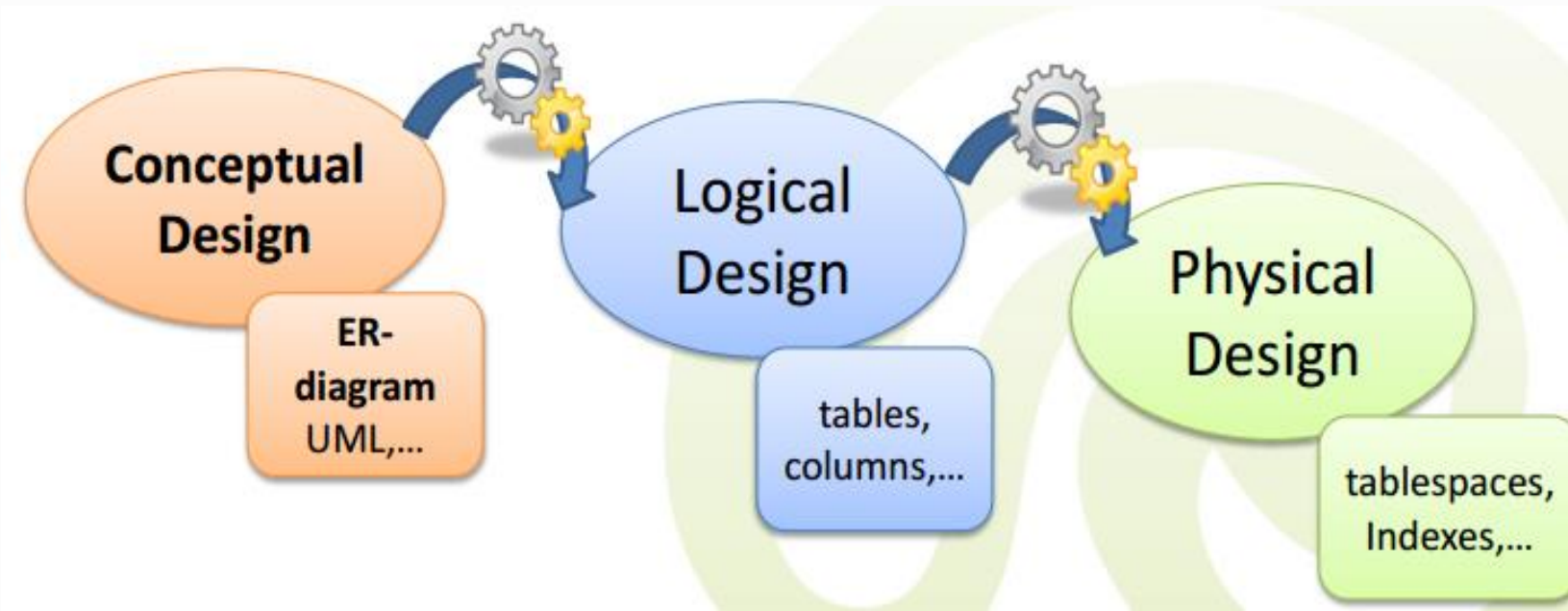
e.g. table spaces, indexes, access paths

depends on used hardware and DBMS software

Phases of DB Design

Modeling the data involves three design phases

- result of one phase is input of the next phase
- often, automatic transition is possible with some additional designer feedback



ER Modeling

Traditional approach to **Conceptual Modeling**

Entity-Relationship Models (ER-Models) also known as Entity-Relationship Diagrams (ERD)

- introduced in 1976 by Peter Chen
- graphical representation

Top-Down-Approach for modeling

entities and attributes

relationships

constraints

Entities

Entities

An entity is an object that exists and is distinguishable from other objects. In other words, the entity can be uniquely identified.

The examples of entities are:

A particular person, for example Dr. Abdul Kalam is an entity.

A particular department, for example Computer Science Department

Entity types

Entity types

An entity type or entity set is a collection of similar entities.

Some examples of entity types are:

All students in SSUET say STUDENT.

All courses in SSUET, say COURSE.

All departments in SSUET, say DEPARTMENT.

An entity may belong to more than one entity type.

For example, a staff working in a particular department can pursue higher education as part-time.

Hence the same person is a LECTURER at one instance and STUDENT a another instance

Entity –Entity Sets

customer-id customer- customer- customer-
 name street city

loan- amount
number

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield

customer

L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300

loan

Attributes

Attributes

a property of an entity, entity type or a relationship type

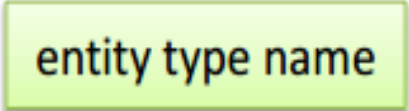
e.g. name of an employee, color of a car, balance of an account, location of a house

attributes can be classified as being:

- *simple or composite*
- *single-valued or multi-valued*
- *stored or derived*

ER Modeling

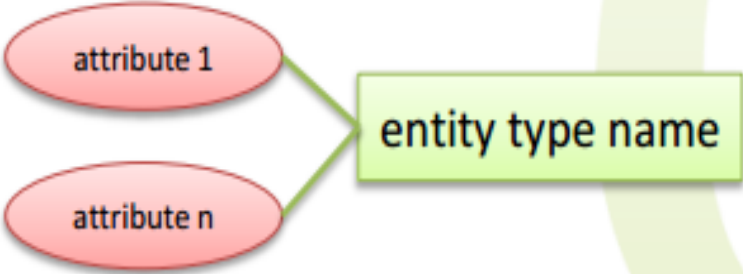
– entity type



entity type name

- Rectangle labeled with the name of the entity
- Usually, name starts with capital letters

– attributes



attribute 1

attribute n

entity type name

- Oval labeled with the name of the attribute
- Usually, name starts with lower case letters

ER Modeling

Textual Representation

Entity types

written: `entity_type_name(attribute_1, ..., attribute_n)`

Entity

written: `(value of attribute_1, ..., value of attribute_n)`

Example

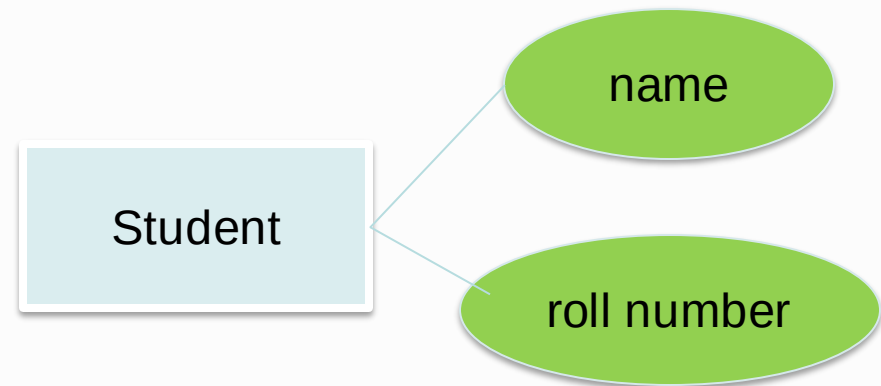
Entity Type Student

`Student(name, rollnumber)`

Entity Set Student

(Ali, 001)

(Sara, 005)



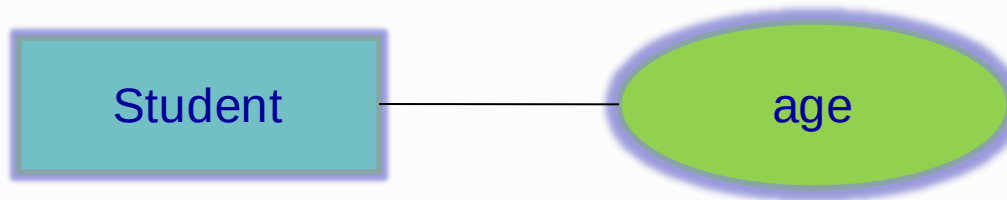
Simple Attribute

Simple Attribute

Attribute composed of a single component with an independent existence

The examples of single value attribute are age of a person, Roll number of the student.

Representation of Single Value Attribute in ER Diagram



Composite Attribute

Composite Attribute

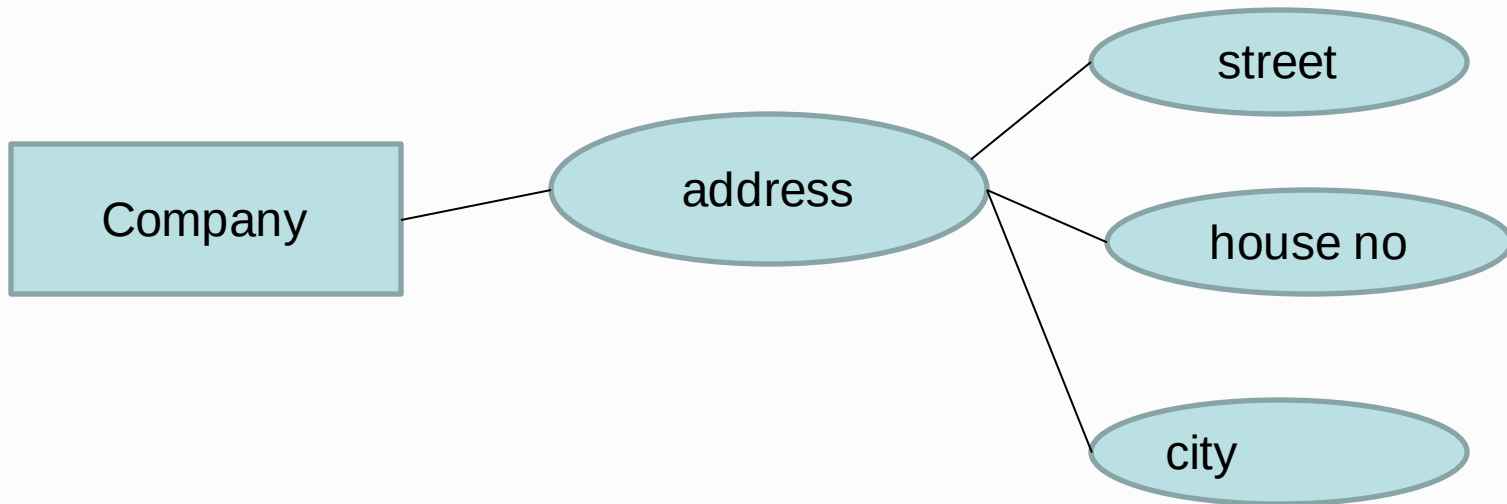
Attribute composed of multiple components, each with an independent existence
graphically represented by connecting sub-attributes to main attribute

textually represented by grouping sub-attributes in ()

e.g. address attribute of a company (is composed of street, house number, ZIP, and city)

Company(address(street, house_no, ZIP, city))

Composite Attribute



Composite Attribute

Composite
Attributes

name

first-name middle-initial last-name

address

street city state postal-code

Component
Attributes

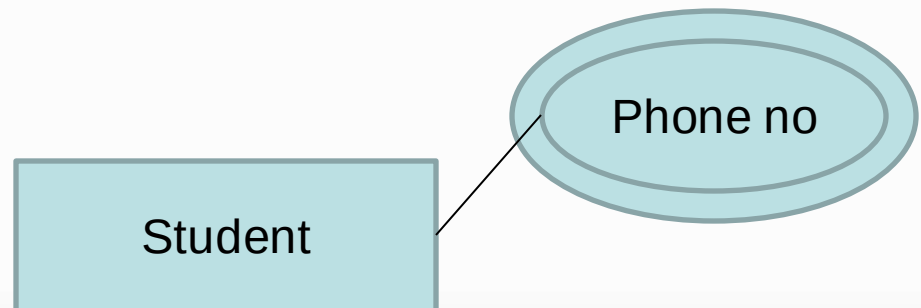
street-number street-name apartment-number

Multi-Valued Attributes

Multi-Valued Attributes (lists)

Attribute holding (possibly) multiple values for each occurrence of an entity type.
graphically indicated by a double-bordered oval
textually represented by enclosing in { }

e.g. telephone number of a student
Student({telephone_no})



Derived Attribute

Stored Attribute

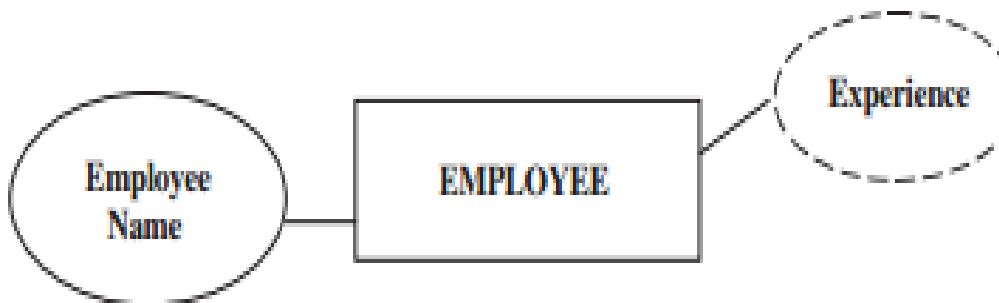
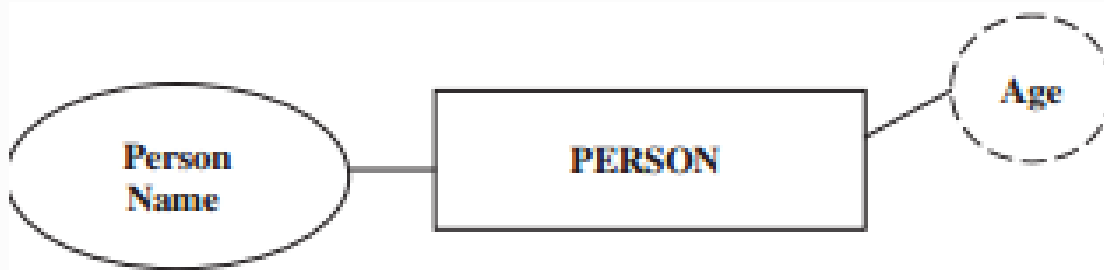
The attribute is directly stored in the database

Derived Attribute

The attribute is (usually) not stored in the DB but derived from an other, stored attribute, indicated by dashed oval

e.g. age can be derived from birth date, average grade can be derived by aggregating all stored grades

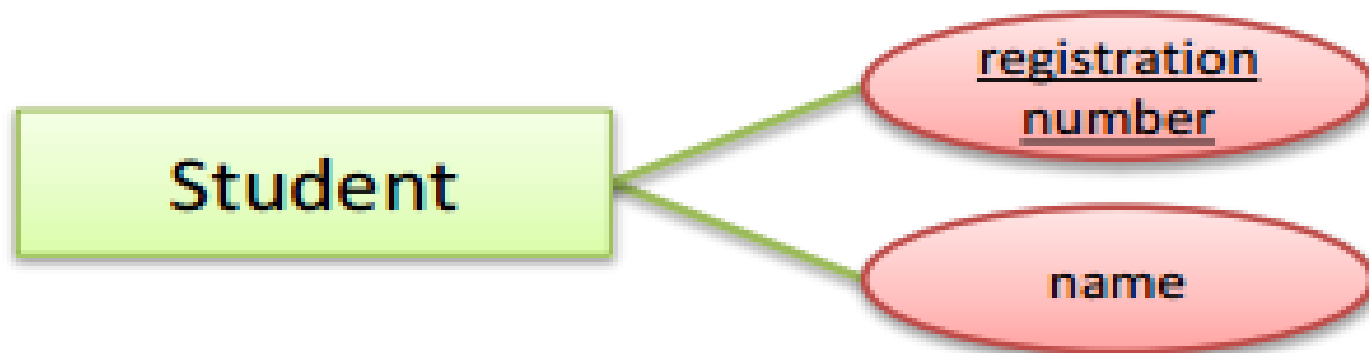
Derived Attribute



Identifier Attribute

Identifier - An attribute (Or combination of attributes) that uniquely identifies individual instances of an entity set

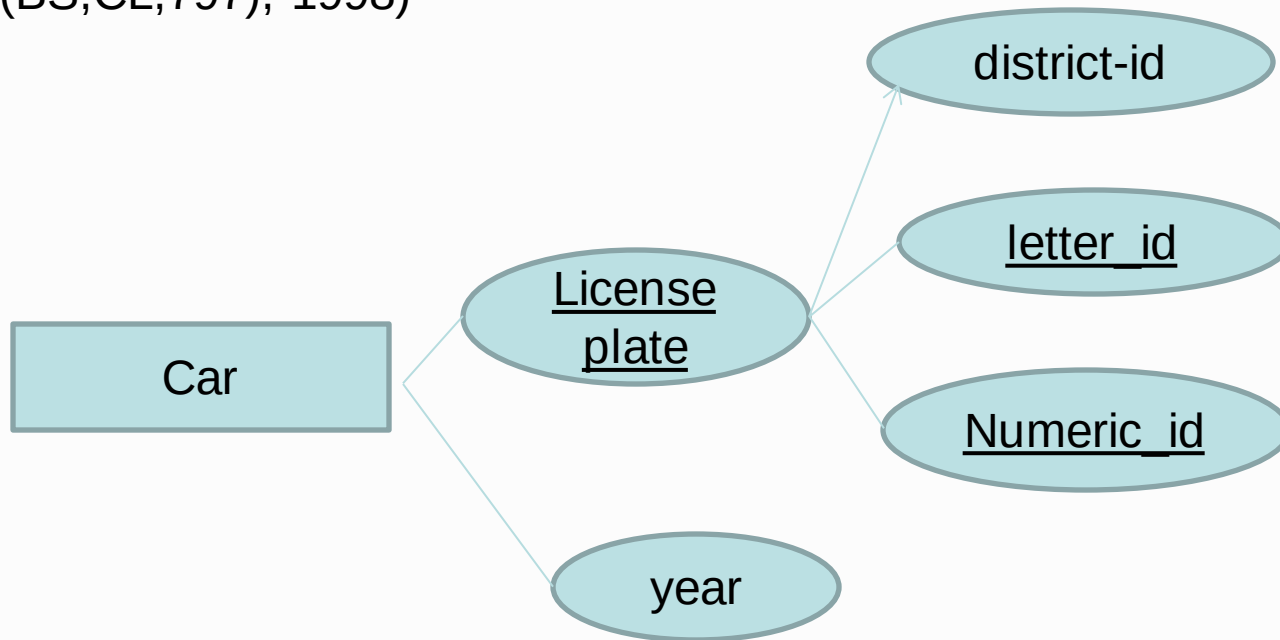
- Also called a key
- Can't same in value
- Can't be null
- Key can be simple or composite



Identifier Attribute

composite key (multiple key attributes)

Car(brand, license_plate(district_id, letter_id, numeric_id), year)
(Toyota, (BS,CL,797), 1998)

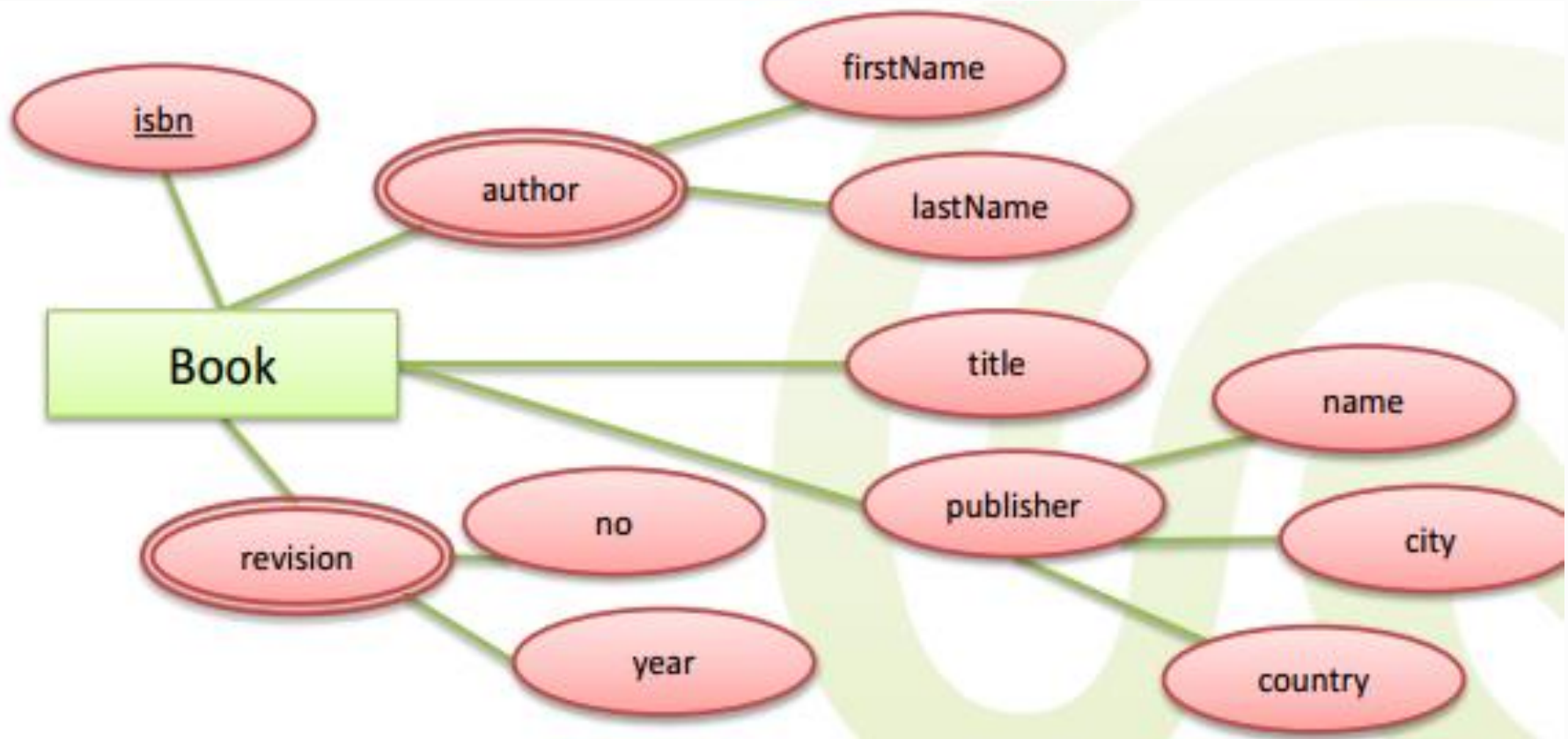


Sample Entity Type

Book(isbn, {author(firstName, lastName)}, title, publisher(name, city, country), {revision(no, year)})

(0321204484, {(Ramez, Elmasri), (Shamkant, Navathe)}, Fundamentals of Database Systems, (Pearson, Boston, US), {(4,2004),(2, 1994)})

Sample Entity Type



ER – Domains

Attributes cannot have arbitrary values: they are restricted by the attribute value sets (domains)

zip codes may be restricted to integer values between

0 and 99999

names may be restricted to character strings with maximum length of 120

- domains are not displayed in ER diagrams
- usually, popular data types are used to describe domains in data modeling
e.g. integer, float, string

ER – Domains

Name	Syntax	description
integer	integer	32-Bit signed integer values between -2^{31} and 2^{31}
double	double	64-Bit floating point values of approximate precision
numeric	numeric(p, s)	A number with p digit before the decimal and s digitals after the decimal
character	char(x)	A textual string of the exact length x
varying character	varchar(x)	A textual string of the maximum length x
date	date	Stores year, month, and day
time	time	Stores hour, minute, and second values

ER – Domains

The original intention of domains was modeling all valid values for an attribute
color: {Red, Blue, Green, Yellow}

NULL Value

Sometimes, an attribute value is not known or an attribute does not apply for an entity this is denoted by the special value NULL so called NULL-value

Example

In application forms, there is one column called phone no. if a person do not have phone then a null value is entered in that column

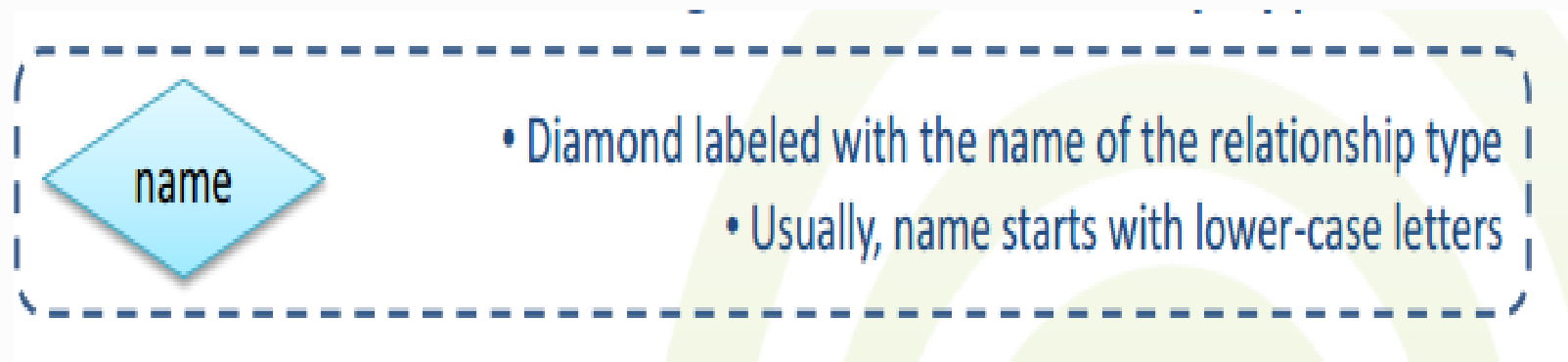
NULL Value

What does it mean when you encounter a NULL value?

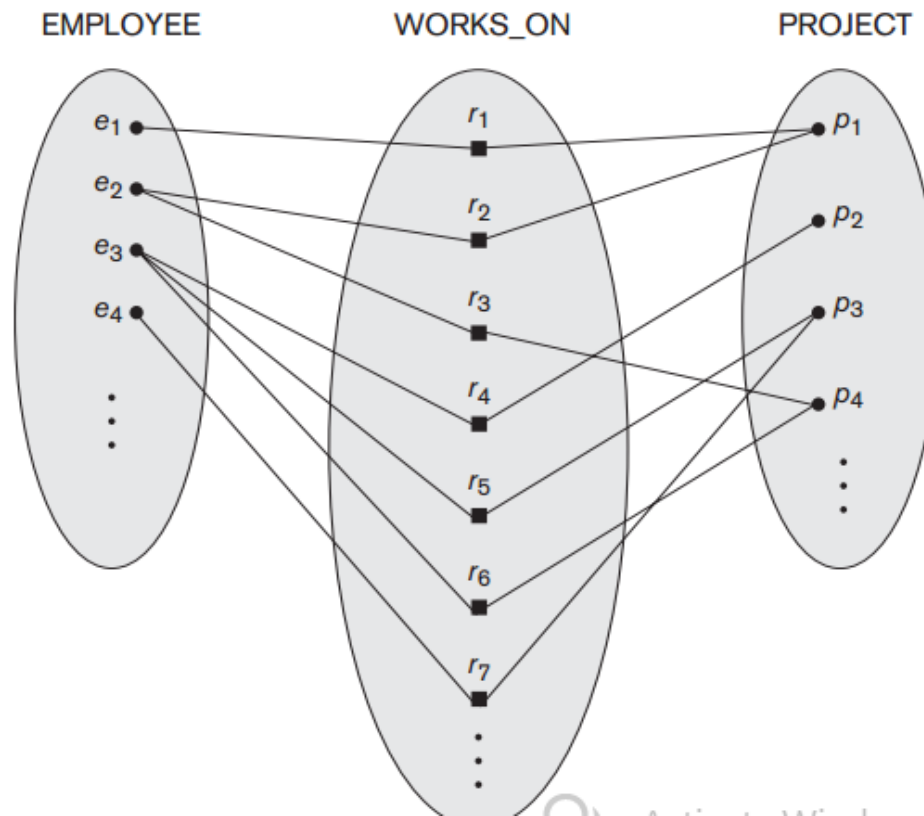
- attribute is not applicable
e.g. attribute middle name when you don't have one
- value is not known
- value will be filled in later
- value is not important for the current entity
- value was just forgotten to set

ER-Relationships

- In ER, there can be relationships between entities
- A relationship type R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations—or a relationship set—among entities from these entity types
- Each relationship instance has a degree i.e. the number of entities it relates to a relationship instance may have attributes



ER-Relationships



ER-Relationships

There is an ownership relation between cats and persons but more modeling detail is needed

Does every person own a cat?

Does every cat have an owner?

Can a cat have multiple owners or a person own multiple cats?

since when does a person own some cat?

who owns whom?

ER-Relationships

- Additionally, restrictions on the combinations of entities participating in an entity set are needed

cardinality annotations are used for this

Cardinality Annotations

Common Cardinality Expressions

(0, 1): each entity may participate at most once in the relationship
(i.e. relationship participation is optional)

(1, 1): each entity is bound exactly once

(0, *): each entity may participate arbitrary often in the relationship

(2, *): each entity may participate arbitrary often in the relationship, but at least twice

Cardinality Annotations

no annotation is usually interpreted as $(0, *)$

if only one symbol / number s is used, this is interpreted as $(0, s)$

$4 = (0, 4)$

sometimes, N or M are used instead of $*$

Cardinality Constraints

Relationships have maximum cardinality and minimum cardinality associated with them which are the maximum and minimum number of entity instances involved in the relationship respectively.

Maximum Cardinality (or called Cardinality)

- > The maximum number of instances of one entity set that may be associated with each instance of another entity set
- > The maximum number is either one or many
 - One represented by a line
 - Many represented by a crow foot

Cardinality Constraints

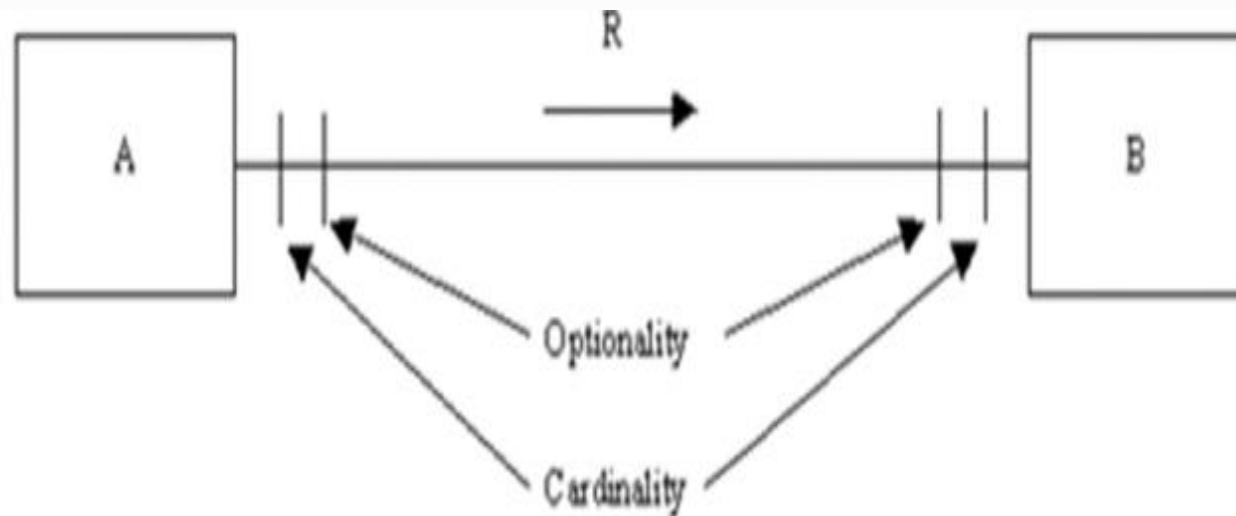
Minimum Cardinality (or called Participation)

- >The minimum number of instances of one entity set that maybe associated with each instance of another entity set
- >The minimum number is either 0 or 1

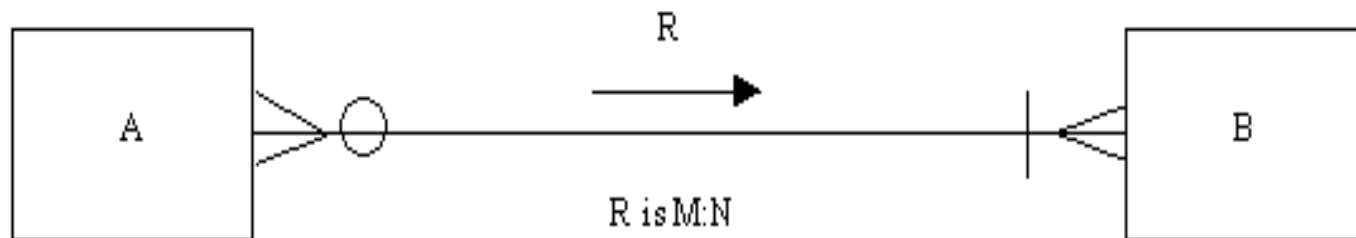
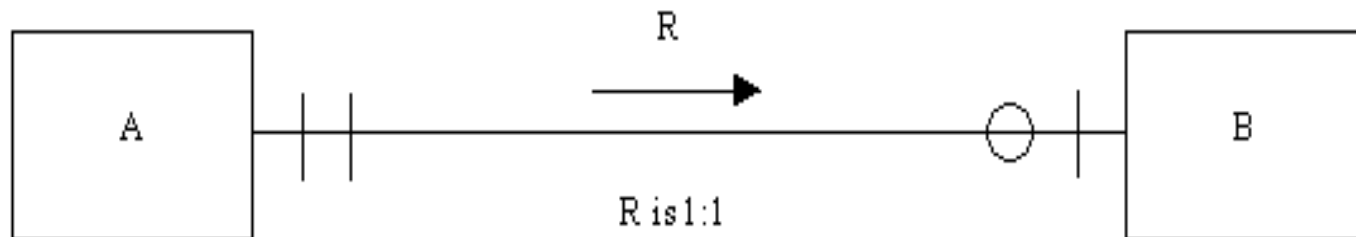
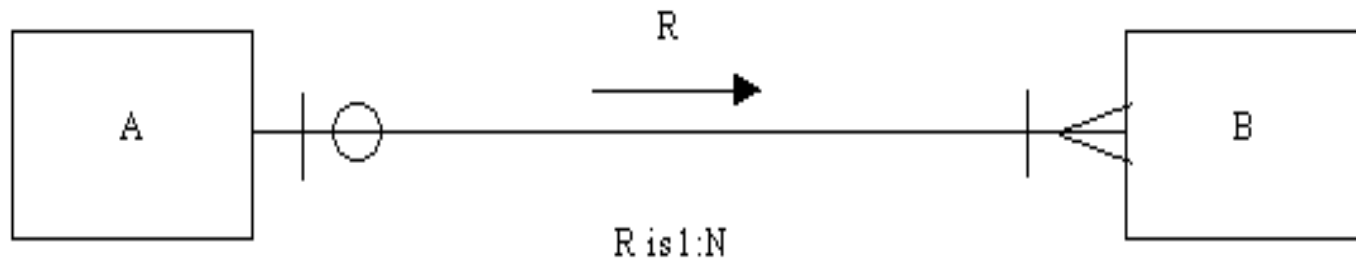
If minimum is zero, means optional

If minimum is one, means mandatory

Cardinality Constraints



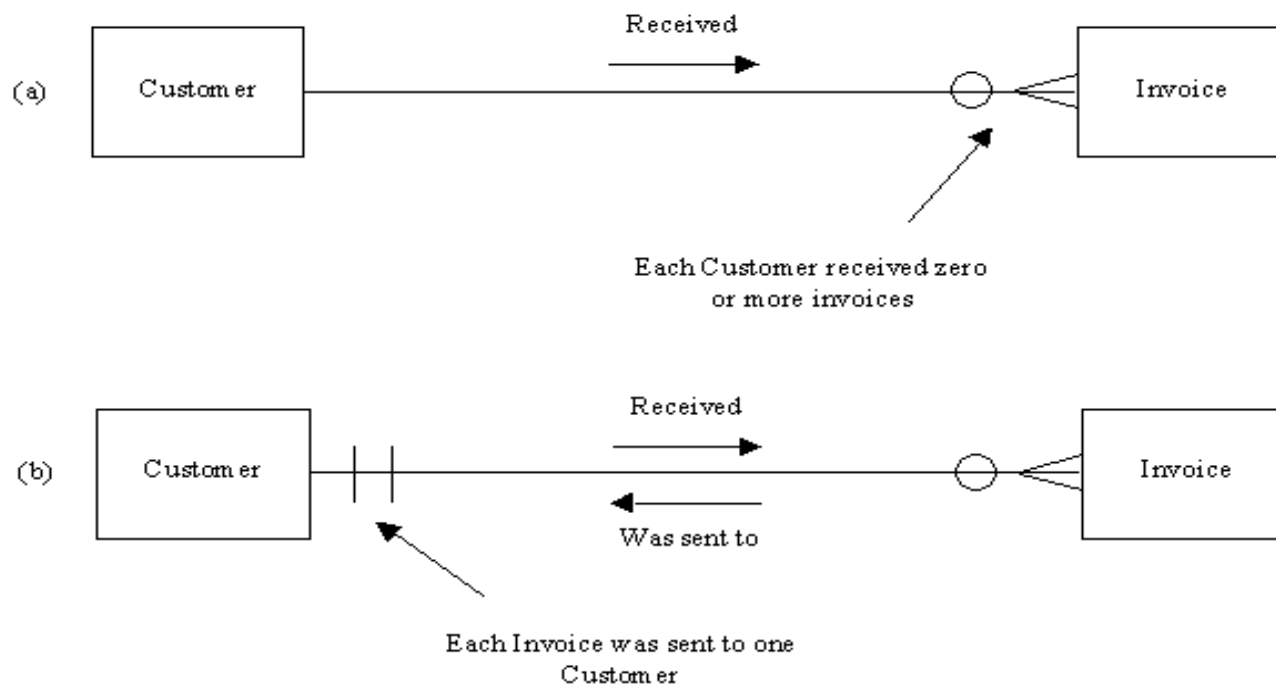
Cardinality Constraints



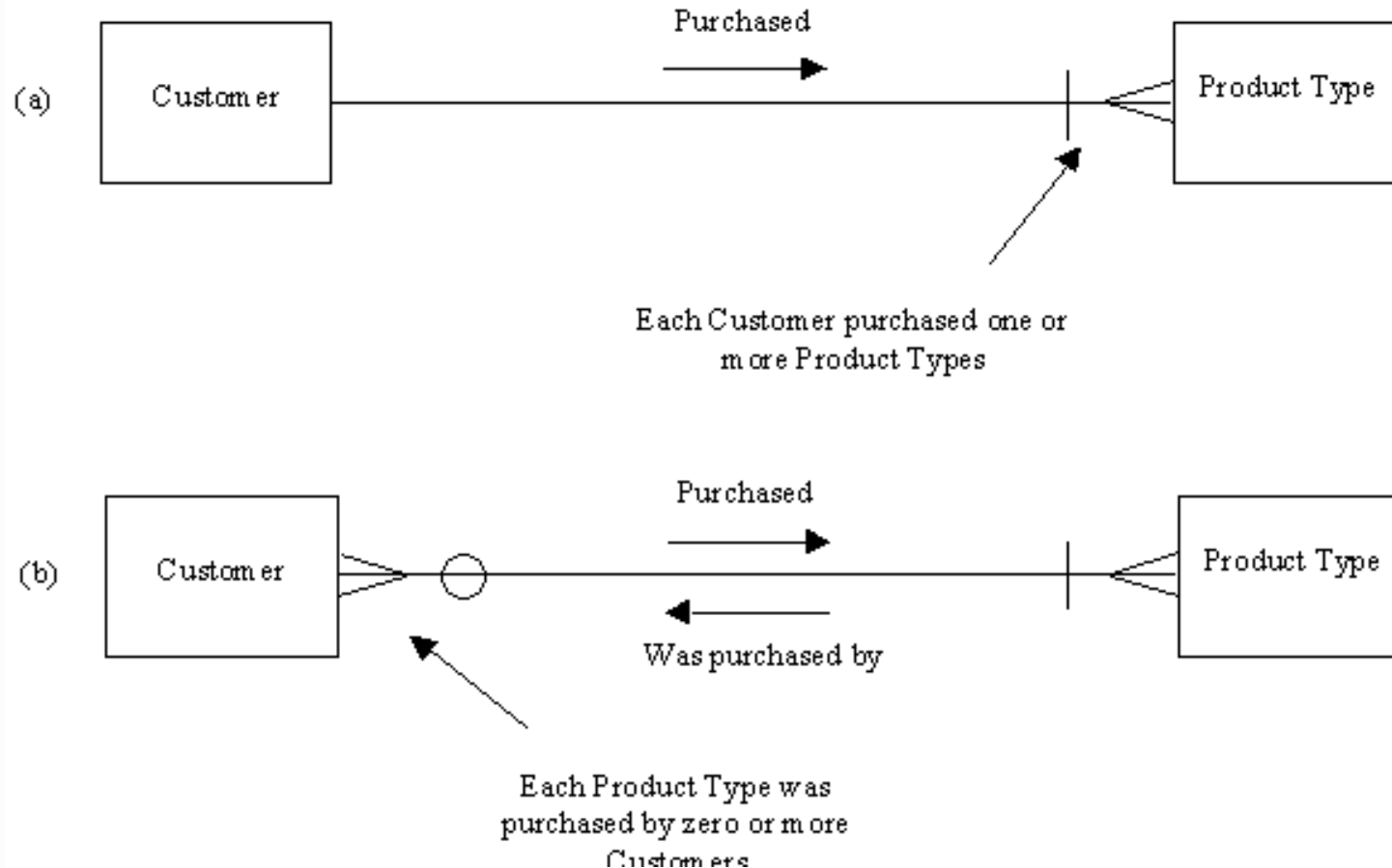
Cardinality Constraints

Cardinality / Optionality	Minimum Instances	Maximum Instances	Notation
Exactly 1 / Mandatory	1	1	
One or More / Mandatory	1	Many	
Zero or One / Optional	0	1	
Zero, One, or Many / Optional	0	Many	

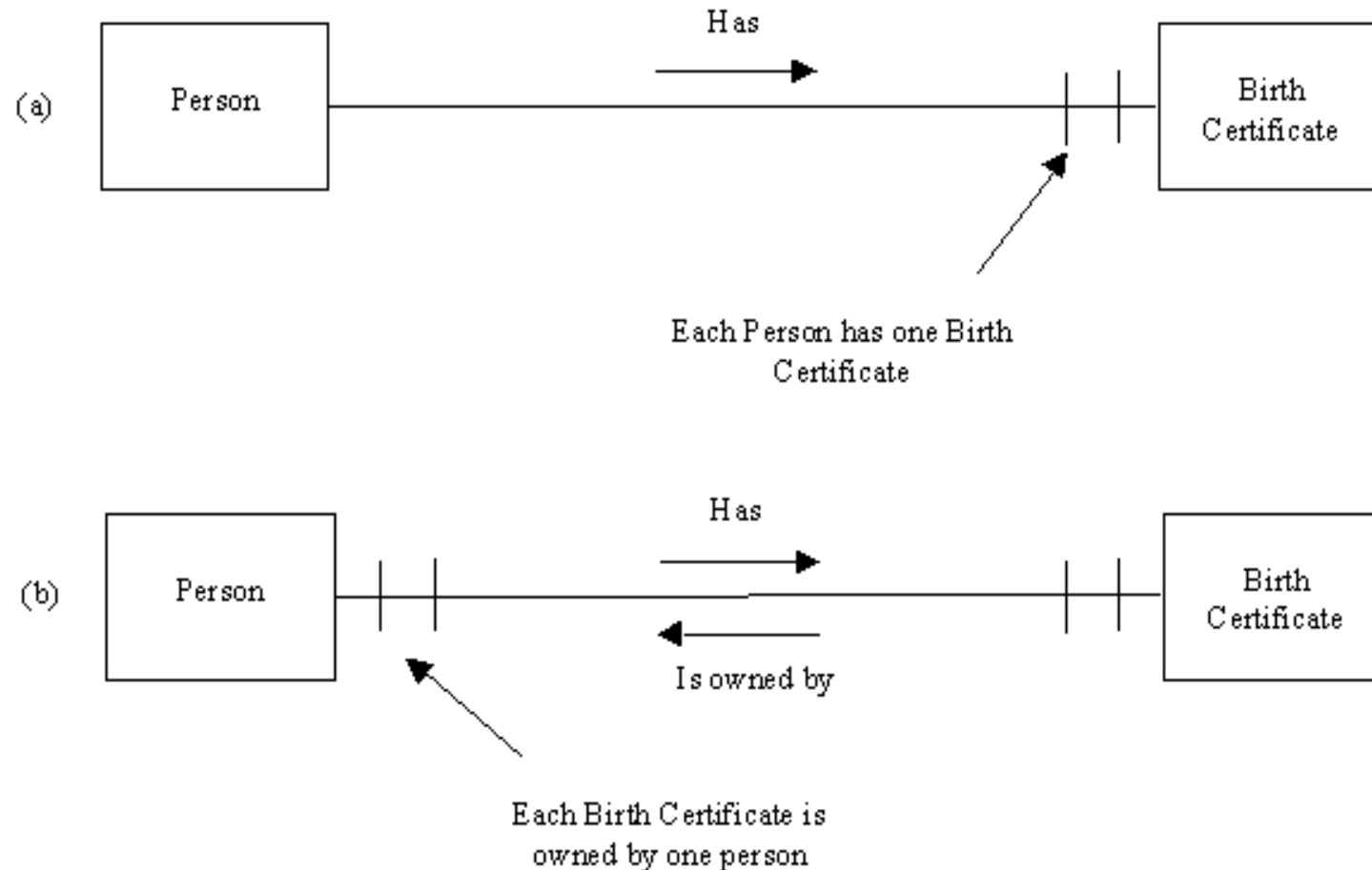
Deriving a One-Many relationship type



Deriving a Many-Many relationship type



Deriving a One-One relationship type



ER-Relationship Degree

Relationship degree refers to the number of associated entities.

The relationship degree can be broadly classified into unary, binary, and ternary relationship

Unary Relationship

The unary relationship is otherwise known as recursive relationship. In the unary relationship the number of associated entity is one.

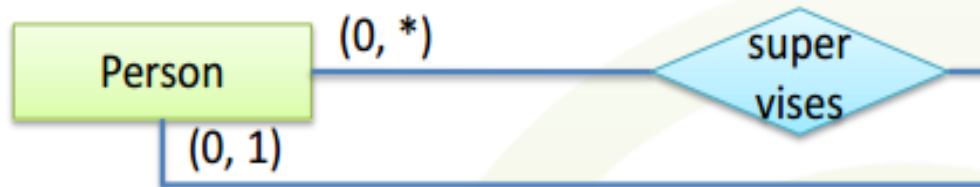
An entity related to itself is known as recursive relationship

Roles and Recursive Relation

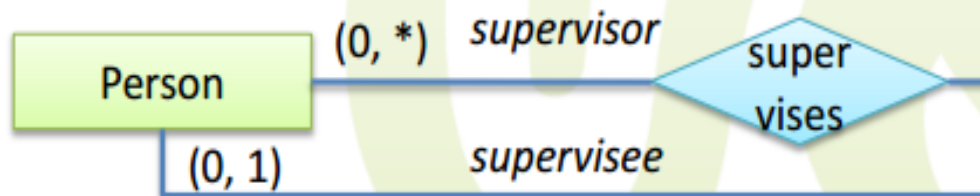
When an entity sets appear in more than one relationship, it is useful to add labels to connecting lines. These labels are called as roles.

ER-Relationship Degree

- e.g. relationship *supervises*



- what is meant? Who is the supervisor? Who is the supervised person?
- roles can be annotated on the relationship lines



ER-Relationship Degree

Binary Relationship

In a binary relationship, two entities are involved.

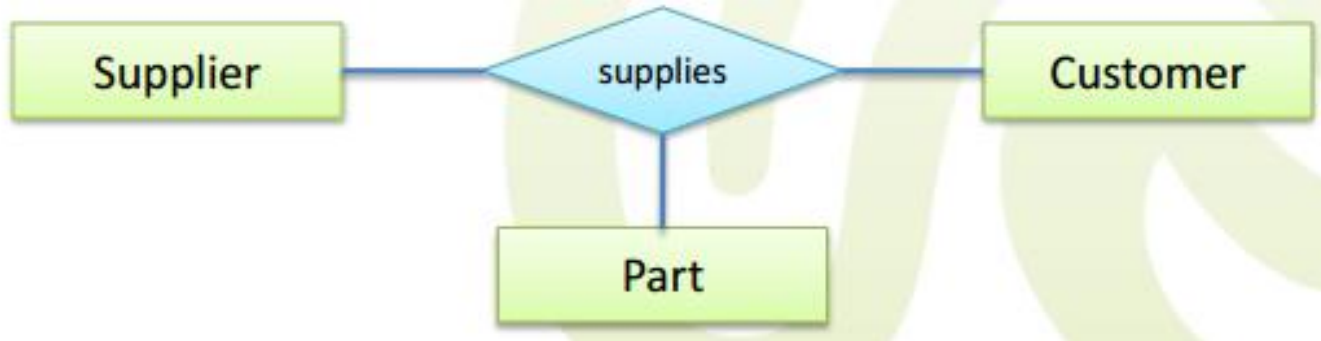


ER-Relationship Degree

Ternary Relationship

In a ternary relationship, three entities are simultaneously involved.

Ternary relationships are required when binary relationships are not sufficient to accurately describe the semantics of an association among three entities.



Participation

Total participation(indicated by double line):

every entity in the entity set participates in at least one relationship in the relationship set

E.g. participation of loan in borrower is total every loan must have a customer associated to it via borrower

Partial participation:

some entities may not participate in any relationship in the relationship set

E.g. participation of customer in borrower is partial

Participation

- To express that all entities of an entity type appear in a certain relationship set, the concept of total participation can be used
- the entity type which is totally participating is indicated by a double line
 - e.g. Each driver's license must belong to an exact person



Weak Entity

An entity set that does not have a primary key is referred to as a weak entity set.

The existence of a weak entity set depends on the existence of a strong entity set; it must relate to the strong set via a one-to-many relationship set.

The discriminator(or partial key) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.

The primary key of a weak entity set is formed by the primary key of the strong entity set on which the weak entity set is existence dependent, plus the weak entity set's discriminator.

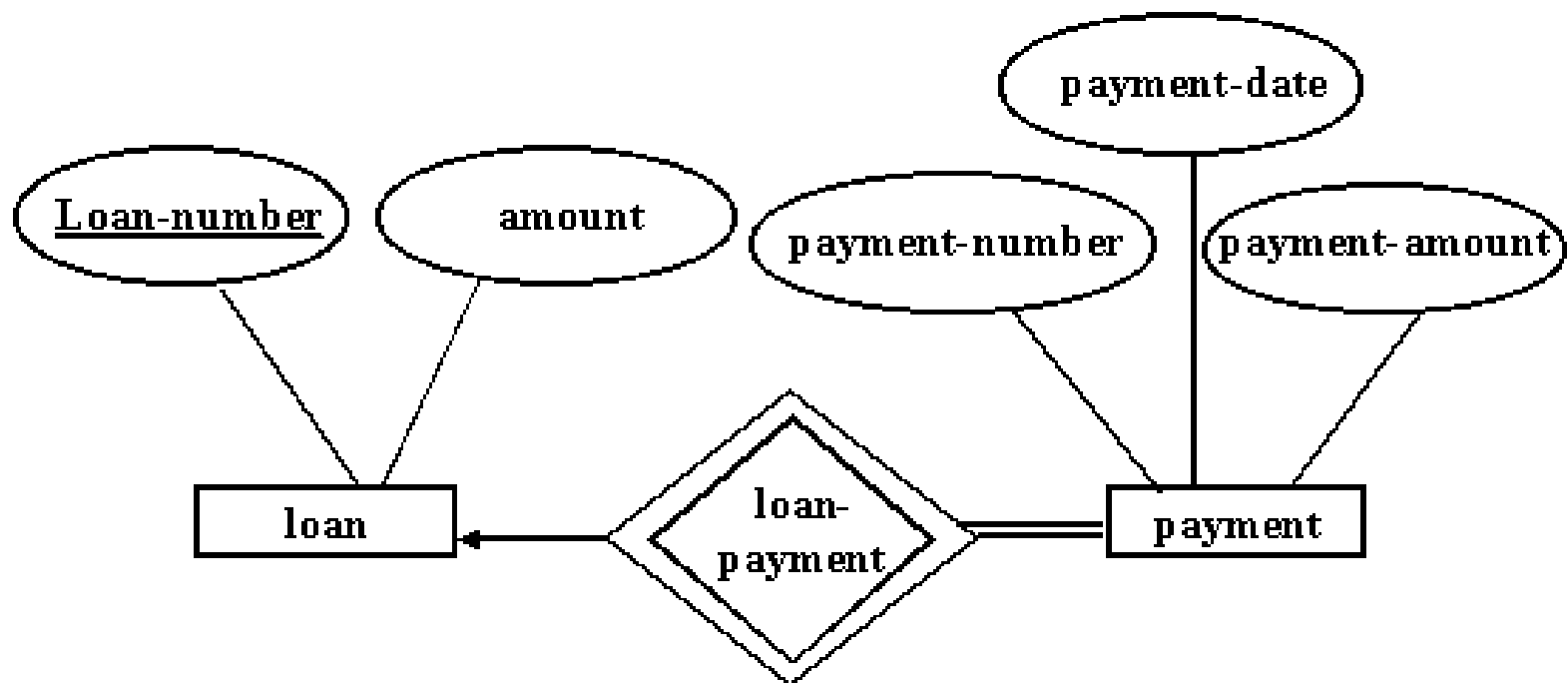
Weak Entity

A weak entity set represented by double rectangles.

The discriminator of a weak entity set represented with a dashed line.

- payment-number—discriminator of the payment entity set
- Primary key for payment—(loan-number, payment-number)

WEAK ENTITY SET



Weak Entity

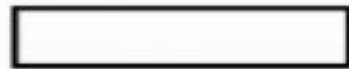
– Example

- An online shopping order contains several order items.



- an *order item* can only exist within an order
- each *order item* can be identified by the order no of it's owning order **and** its item line

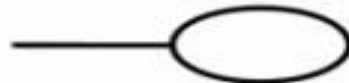
[illegible]



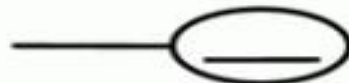
Entity Type



Relationship Type



Attribute



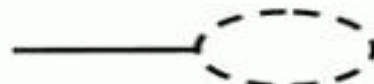
Key Attribute



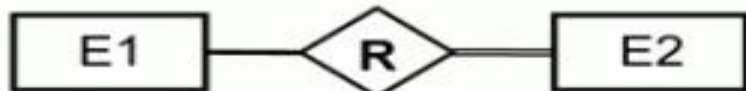
Multivalued Attribute



Composite Attribute



Derived Attribute



Total Participation of E2 in **R**



Ratio Cardinality 1:N for E1 **R** E2